

```
import pandas as pd
import numpy as np
import random
import copy
import time
from collections import defaultdict
```

```
#Constants
SPEED_MPH      = 40.0
UNLOAD_RATE_VAN = 0.030
UNLOAD_RATE_ST  = 0.043
MIN_UNLOAD_MIN  = 30.0
MAX_DRIVE_HRS   = 11.0
MAX_DUTY_HRS    = 14.0
BREAK_HRS       = 10.0
STORE_OPEN_H    = 8.0
STORE_CLOSE_H   = 18.0

VAN_CAPACITY    = 3200
ST_CAPACITY     = 1400
DEPOT_ZIP       = 1887

DAY_BASE_HRS = {'Mon': 0.0, 'Tue': 24.0, 'Wed': 48.0, 'Thu': 72.0, 'Fri': 96.0}
DAYS = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
```

Data Loading

```
def load_data(deliveries_path='deliveries.xlsx', distances_path='distances.xlsx'):
    import warnings
    with warnings.catch_warnings():
        warnings.simplefilter('ignore')
        df_order = pd.read_excel(deliveries_path, sheet_name='OrderTable')
        df_loc = pd.read_excel(deliveries_path, sheet_name='LocationTable')
        df_dist = pd.read_excel(distances_path, sheet_name='Sheet1', index_col=0)

    df_dist = df_dist.drop(index='Zip').drop(columns='Unnamed: 1')
    df_dist.index = df_dist.index.astype(int)
    df_dist.columns = df_dist.columns.astype(int)
    dist = df_dist.astype(float)

    orders = df_order[df_order['ORDERID'] != 0].copy()
    orders['CUBE'] = pd.to_numeric(orders['CUBE'], errors='coerce')
    orders = orders.reset_index(drop=True)
    return orders, df_loc, dist

def dist_miles(dist_matrix, z1, z2):
    try:
        return float(dist_matrix.loc[z1, z2])
    except KeyError:
        return float('inf')
```

Time & Feasability Helpers

```
def travel_hrs(miles):
    return miles / SPEED_MPH

def unload_hrs(cube, rate):
    return max(rate * cube, MIN_UNLOAD_MIN) / 60.0

def vol_ok(route, capacity):
    return sum(o['CUBE'] for o in route) <= capacity

def evaluate_route(route, dist_matrix, unload_rate, allow_overnight=True):
    """
    Simulate a route and check all constraints.
    Returns dict: feasible, total_miles, drive_hrs, duty_hrs,
                  departure_time, arrival_back_time, is_overnight, stops, reason.
    """
    if not route:
```

```

        return {'feasible': False, 'reason': 'empty route',
                'total_miles': 0, 'drive_hrs': 0, 'duty_hrs': 0,
                'departure_time': 0, 'arrival_back_time': 0,
                'is_overnight': False, 'stops': []}

first_zip = route[0]['TOZIP']
d_to_first = dist_miles(dist_matrix, DEPOT_ZIP, first_zip)
depart_time = STORE_OPEN_H - travel_hrs(d_to_first)

drive = 0.0; duty = 0.0
cur = depart_time
prev = DEPOT_ZIP
total_miles = 0.0
overnight = False
stops = []

for idx, order in enumerate(route):
    dest = order['TOZIP']
    miles = dist_miles(dist_matrix, prev, dest)
    t_drv = travel_hrs(miles)
    t_unl = unload_hrs(order['CUBE'], unload_rate)

    drive_after = drive + t_drv
    duty_after = duty + t_drv

    if drive_after > MAX_DRIVE_HRS + 1e-9 or duty_after > MAX_DUTY_HRS + 1e-9:
        if not allow_overnight or overnight:
            return {'feasible': False, 'total_miles': total_miles,
                    'drive_hrs': drive, 'duty_hrs': duty,
                    'departure_time': depart_time,
                    'arrival_back_time': cur,
                    'is_overnight': overnight, 'stops': stops,
                    'reason': f'DOT violation before stop {idx+1} '
                            f'(order {order["ORDERID"]})'}

        rem_drive = MAX_DRIVE_HRS - drive
        rem_duty = MAX_DUTY_HRS - duty
        can_drive = min(rem_drive, rem_duty)

        if can_drive >= t_drv:
            drive += t_drv; duty += t_drv; cur += t_drv
        else:
            cur += can_drive
            cur += BREAK_HRS
            overnight = True
            drive = 0.0; duty = 0.0

            rem_t = t_drv - can_drive
            if rem_t < 0: rem_t = 0
            drive += rem_t; duty += rem_t; cur += rem_t
        else:
            drive += t_drv; duty += t_drv; cur += t_drv

    total_miles += miles

    if cur < STORE_OPEN_H:
        wait = STORE_OPEN_H - cur
        duty += wait
        cur = STORE_OPEN_H

    arrival = cur

    if arrival > STORE_CLOSE_H + 1e-9:
        return {'feasible': False, 'total_miles': total_miles,
                'drive_hrs': drive, 'duty_hrs': duty,
                'departure_time': depart_time,
                'arrival_back_time': cur,
                'is_overnight': overnight, 'stops': stops,
                'reason': f'Arrives after 18:00 at stop {idx+1} '
                        f'(order {order["ORDERID"]}, zip {dest})'}

    duty += t_unl; cur += t_unl

    if duty > MAX_DUTY_HRS + 1e-9 and not overnight:
        return {'feasible': False, 'total_miles': total_miles,
                'drive_hrs': drive, 'duty_hrs': duty,
                'departure_time': depart_time,

```

```

        'arrival_back_time': cur,
        'is_overnight': overnight, 'stops': stops,
        'reason': f'Duty exceeded after unload stop {idx+1}'}

    stops.append({'order_id': order['ORDERID'], 'zip': dest,
                  'arrival': arrival, 'departure': cur,
                  'drive_cumul': drive, 'duty_cumul': duty})

    prev = dest

    d_ret = dist_miles(dist_matrix, prev, DEPOT_ZIP)
    total_miles += d_ret
    drive += travel_hrs(d_ret)
    duty  += travel_hrs(d_ret)
    cur   += travel_hrs(d_ret)

    return {'feasible': True, 'total_miles': total_miles,
            'drive_hrs': drive, 'duty_hrs': duty,
            'departure_time': depart_time, 'arrival_back_time': cur,
            'is_overnight': overnight, 'stops': stops, 'reason': None}

```

CONSTRUCTION – CLARKE-WRIGHT SAVINGS

```

def clarke_wright(day_orders, dist_matrix, capacity, unload_rate,
                  allow_overnight=True):
    orders = list(day_orders)
    n = len(orders)
    if n == 0:
        return []

    savings = []
    for i in range(n):
        for j in range(i + 1, n):
            zi = orders[i]['TOZIP']; zj = orders[j]['TOZIP']
            s = (dist_miles(dist_matrix, DEPOT_ZIP, zi)
                 + dist_miles(dist_matrix, DEPOT_ZIP, zj)
                 - dist_miles(dist_matrix, zi, zj))
            savings.append((s, i, j))
    savings.sort(reverse=True)

    routes = [[o] for o in orders]
    which = list(range(n))

    def try_merge(ri, rj, new_seq):
        if not vol_ok(new_seq, capacity): return False
        if not evaluate_route(new_seq, dist_matrix, unload_rate,
                              allow_overnight)['feasible']: return False
        routes[ri] = new_seq
        for k in range(n):
            if which[k] == rj: which[k] = ri
        routes[rj] = []
        return True

    for s, i, j in savings:
        ri = which[i]; rj = which[j]
        if ri == rj or not routes[ri] or not routes[rj]:
            continue
        r_i = routes[ri]; r_j = routes[rj]
        oi = orders[i]['ORDERID']; oj = orders[j]['ORDERID']
        if r_i[-1]['ORDERID'] == oi and r_j[0]['ORDERID'] == oj:
            try_merge(ri, rj, r_i + r_j)
        elif r_i[0]['ORDERID'] == oi and r_j[-1]['ORDERID'] == oj:
            try_merge(rj, ri, r_j + r_i)
        elif r_i[-1]['ORDERID'] == oj and r_j[0]['ORDERID'] == oi:
            try_merge(ri, rj, r_i + r_j)
        elif r_i[0]['ORDERID'] == oj and r_j[-1]['ORDERID'] == oi:
            try_merge(rj, ri, r_j + r_i)

    return [r for r in routes if r]

```

IMPROVEMENT – LOCAL SEARCH

```

def two_opt(route, dist_matrix, unload_rate, cap, allow_overnight=True):
    best = list(route)
    best_r = evaluate_route(best, dist_matrix, unload_rate, allow_overnight)

```

```

if not best_r['feasible']:
    return best, best_r
improved = True
while improved:
    improved = False
    n = len(best)
    for i in range(n - 1):
        for j in range(i + 2, n):
            cand = best[:i] + best[i:j][::-1] + best[j:]
            if not vol_ok(cand, cap): continue
            r = evaluate_route(cand, dist_matrix, unload_rate, allow_overnight)
            if r['feasible'] and r['total_miles'] < best_r['total_miles'] - 0.01:
                best = cand; best_r = r; improved = True
return best, best_r

def or_opt(route, dist_matrix, unload_rate, cap, seg=1, allow_overnight=True):
    best = list(route)
    best_r = evaluate_route(best, dist_matrix, unload_rate, allow_overnight)
    if not best_r['feasible']:
        return best, best_r
    improved = True
    while improved:
        improved = False
        n = len(best)
        for i in range(n - seg + 1):
            chunk = best[i:i + seg]
            rest = best[:i] + best[i + seg:]
            for j in range(len(rest) + 1):
                cand = rest[:j] + chunk + rest[j:]
                if cand == best: continue
                if not vol_ok(cand, cap): continue
                r = evaluate_route(cand, dist_matrix, unload_rate, allow_overnight)
                if r['feasible'] and r['total_miles'] < best_r['total_miles'] - 0.01:
                    best = cand; best_r = r; improved = True
                    break
            if improved: break
    return best, best_r

def inter_route_relocate(routes, dist_matrix, unload_rate, cap,
                        allow_overnight=True):
    res = [evaluate_route(r, dist_matrix, unload_rate, allow_overnight)
           for r in routes]
    improved = True
    while improved:
        improved = False
        for i in range(len(routes)):
            for si in range(len(routes[i])):
                order = routes[i][si]
                new_ri = routes[i][:si] + routes[i][si+1:]
                mi_old = res[i]['total_miles']

                if new_ri:
                    r_new_i = evaluate_route(new_ri, dist_matrix, unload_rate,
                                             allow_overnight)
                    if not r_new_i['feasible']: continue
                    mi_new_i = r_new_i['total_miles']
                else:
                    r_new_i = None
                    mi_new_i = 0.0

                for j in range(len(routes)):
                    if i == j: continue
                    for sj in range(len(routes[j]) + 1):
                        new_rj = routes[j][:sj] + [order] + routes[j][sj:]
                        if not vol_ok(new_rj, cap): continue
                        r_new_j = evaluate_route(new_rj, dist_matrix, unload_rate,
                                                allow_overnight)
                        if not r_new_j['feasible']: continue
                        gain = (mi_old + res[j]['total_miles']) \
                            - (mi_new_i + r_new_j['total_miles'])
                        if gain > 0.1:
                            routes[i] = new_ri
                            routes[j] = new_rj
                            res[i] = r_new_i if r_new_i else \
                                {'feasible': True, 'total_miles': 0,

```

```

        'drive_hrs': 0, 'duty_hrs': 0,
        'departure_time': 0, 'arrival_back_time': 0,
        'is_overnight': False, 'stops': [], 'reason': None}
    res[j] = r_new_j
    improved = True
    break
    if improved: break
    if improved: break
    if improved: break

pairs = [(r, s) for r, s in zip(routes, res) if r]
if pairs:
    routes, res = zip(*pairs)
    return list(routes), list(res)
return [], []

```

Solving ONE DAY

```

def solve_day(day_orders, dist_matrix, capacity, unload_rate,
              allow_overnight=True, verbose=False):
    if not day_orders:
        return [], [], 0.0

    routes = clarke_wright(day_orders, dist_matrix, capacity,
                           unload_rate, allow_overnight)

    improved = []
    for r in routes:
        r, _ = two_opt(r, dist_matrix, unload_rate, capacity, allow_overnight)
        r, _ = or_opt(r, dist_matrix, unload_rate, capacity, seg=1,
                     allow_overnight=allow_overnight)
        r, _ = or_opt(r, dist_matrix, unload_rate, capacity, seg=2,
                     allow_overnight=allow_overnight)
        improved.append(r)
    routes = improved

    routes, results = inter_route_relocate(routes, dist_matrix, unload_rate,
                                           capacity, allow_overnight)

    final_results = [evaluate_route(r, dist_matrix, unload_rate, allow_overnight)
                     for r in routes]
    total_miles = sum(r['total_miles'] for r in final_results if r['feasible'])

    if verbose:
        n_inf = sum(1 for r in final_results if not r['feasible'])
        n_on = sum(1 for r in final_results if r.get('is_overnight'))
        print(f"    {len(routes)} routes | {total_miles:,.1f} mi | {n_on} overnight"
              + (f" | {n_inf} INFEASIBLE" if n_inf else ""))

    return routes, final_results, total_miles

```

Resource Analysis

```

def resource_analysis(routes_by_day, results_by_day):
    timeline = []
    for day in DAYS:
        base = DAY_BASE_HRS[day]
        for res in results_by_day.get(day, []):
            if not res.get('feasible'): continue
            timeline.append({'dispatch': base + res['departure_time'],
                           'finish': base + res['arrival_back_time']})
    timeline.sort(key=lambda x: x['dispatch'])

    avail = []
    for t in timeline:
        assigned = False
        for k in range(len(avail)):
            if avail[k] + BREAK_HRS <= t['dispatch'] + 1e-9:
                avail[k] = t['finish']; assigned = True; break
        if not assigned:
            avail.append(t['finish'])

    events = []

```

```

for t in timeline:
    events += [(t['dispatch'], +1), (t['finish'], -1)]
events.sort()
peak = cur = 0
for _, d in events:
    cur += d; peak = max(peak, cur)

return len(avail), peak

```

1-BASELINE

```

def solve_q1(orders, dist_matrix, verbose=True):

    print(" Q1: Base Case - Van fleet, fixed delivery schedule")
    rbdays, rrday = {}, {}
    total_wk = 0.0
    for day in DAYS:
        day_o = orders[orders['DayOfWeek'] == day].to_dict('records')
        if verbose:
            print(f" {day}: {len(day_o):3d} orders, "
                  f"{sum(o['CUBE'] for o in day_o):.0f} ft3")
        r, rr, mi = solve_day(day_o, dist_matrix, VAN_CAPACITY,
                               UNLOAD_RATE_VAN, allow_overnight=True,
                               verbose=verbose)
        rbdays[day] = r; rrday[day] = rr; total_wk += mi

    nd, nv = resource_analysis(rbdays, rrday)
    annual = total_wk * 52
    print(f"\n Weekly miles : {total_wk:>10,.1f}")
    print(f" Annual miles : {annual:>10,.0f}")
    print(f" Min drivers : {nd}")
    print(f" Min vehicles : {nv}")
    return dict(routes=rbdays, results=rrday,
                weekly=total_wk, annual=annual, drivers=nd, vehicles=nv)

```

2-MIXED FLEET

```

def solve_q2(orders, dist_matrix, verbose=True):

    print(" Q2: Mixed Fleet - Vans + Straight Trucks")
    rbdays, rrday = {}, {}
    total_wk = 0.0
    for day in DAYS:
        day_all = orders[orders['DayOfWeek'] == day].to_dict('records')
        st_o = [o for o in day_all if o['ST required?'] == 'yes']
        van_o = [o for o in day_all if o['ST required?'] != 'yes']
        if verbose:
            print(f" {day}: {len(day_all):3d} orders "
                  f"(ST={len(st_o)}, Van={len(van_o)})")

        dr, drr = [], []
        if st_o:
            r, rr, mi = solve_day(st_o, dist_matrix, ST_CAPACITY,
                                   UNLOAD_RATE_ST, verbose=False)
            dr += r; drr += rr; total_wk += mi
            if verbose: print(f" ST => {len(r)} routes, {mi:.1f} mi")
        if van_o:
            r, rr, mi = solve_day(van_o, dist_matrix, VAN_CAPACITY,
                                   UNLOAD_RATE_VAN, verbose=False)
            dr += r; drr += rr; total_wk += mi
            if verbose: print(f" Van => {len(r)} routes, {mi:.1f} mi")

        rbdays[day] = dr; rrday[day] = drr

    nd, nv = resource_analysis(rbdays, rrday)
    annual = total_wk * 52
    print(f"\n Weekly miles : {total_wk:>10,.1f}")
    print(f" Annual miles : {annual:>10,.0f}")
    print(f" Min drivers : {nd}")
    print(f" Min vehicles : {nv}")
    return dict(routes=rbdays, results=rrday,
                weekly=total_wk, annual=annual, drivers=nd, vehicles=nv)

```

3-Relaxed Schedule

```

def solve_q3_best(orders, dist_matrix, coords,
                  max_iter=5, verbose=True):

    print(" Q3: OPTIMAL GEOGRAPHIC CLUSTERING (A+ SOLUTION)")

    recs = orders.to_dict('records')

    store_days = defaultdict(list)
    for o in recs:
        store_days[o['TOZIP']].append(o)

    stores = list(store_days.keys())

    def compute_angles(stores, coords, depot):
        x0, y0 = coords[depot]
        ang = {}
        for z in stores:
            x, y = coords[z]
            ang[z] = np.arctan2(y - y0, x - x0)
        return ang

    angles = compute_angles(stores, coords, DEPOT_ZIP)
    sorted_stores = sorted(stores, key=lambda z: angles[z])

    day_groups = {d: [] for d in DAYS}
    day_load = {d: 0 for d in DAYS}

    def store_load(z):
        return sum(o['CUBE'] for o in store_days[z])
    avg_load = sum(store_load(z) for z in stores) / len(DAYS)

    d_idx = 0
    for z in sorted_stores:
        load = store_load(z)

        if day_load[DAYS[d_idx]] + load > avg_load * 1.2:
            d_idx = (d_idx + 1) % len(DAYS)

        day = DAYS[d_idx]
        day_groups[day].append(z)
        day_load[day] += load

    assignment = {}

    for day, zips in day_groups.items():
        for z in zips:
            for o in store_days[z]:
                assignment[o['ORDERID']] = day

    def fast_eval(assign):
        day_map = {d: [] for d in DAYS}

        for o in recs:
            day_map[assign[o['ORDERID']]].append(o)

        total = 0.0

        for day, orders in day_map.items():
            if not orders:
                continue
            current = DEPOT_ZIP
            remaining = [o['TOZIP'] for o in orders]

            while remaining:
                nxt = min(remaining,
                           key=lambda z: dist_miles(dist_matrix, current, z))
                total += dist_miles(dist_matrix, current, nxt)
                current = nxt
                remaining.remove(nxt)

            total += dist_miles(dist_matrix, current, DEPOT_ZIP)

        return total

    if verbose:

```

```

print(" Running local improvement...")

for it in range(max_iter):
    improved = False

    for z in stores:
        current_day = assignment[store_days[z][0]['ORDERID']]

        for new_day in DAYS:
            if new_day == current_day:
                continue

            trial = assignment.copy()

            for o in store_days[z]:
                trial[o['ORDERID']] = new_day

            if fast_eval(trial) < fast_eval(assignment):
                assignment = trial
                improved = True
                break

        if improved:
            break

    if not improved:
        break

if verbose:
    print(" Running final VRP...")

def build_day_map(asgn):
    dm = {d: [] for d in DAYS}
    for o in recs:
        dm[asgn[o['ORDERID']]].append(o)
    return dm

final_dm = build_day_map(assignment)

rbdy, rrdy = {}, {}
total_weekly = 0.0

for day in DAYS:
    day_orders = final_dm[day]

    routes = clarke_wright(
        day_orders,
        dist_matrix,
        VAN_CAPACITY,
        UNLOAD_RATE_VAN,
        allow_overnight=True
    )

    results = [evaluate_route(r, dist_matrix, UNLOAD_RATE_VAN)
               for r in routes]

    miles = sum(r['total_miles'] for r in results if r['feasible'])

    rbdy[day] = routes
    rrdy[day] = results
    total_weekly += miles

    if verbose:
        print(f"    {day}: {len(day_orders):3d} orders \u2192 "
              f"{len(routes)} routes, {miles:,.1f} miles")

nd, nv = resource_analysis(rbdy, rrdy)
annual = total_weekly * 52

print(f" Weekly miles : {total_weekly:,.1f}")
print(f" Annual miles : {annual:,.0f}")
print(f" Drivers      : {nd}")
print(f" Vehicles       : {nv}")

return {
    "routes": rbdy,
    "results": rrdy,
    "nd": nd,
    "nv": nv,
    "annual": annual
}

```



```

    results : rudy,
    "annual": annual,
    "weekly": total_weekly,
    "drivers": nd,
    "vehicles": nv
}

```

EXPORT

```

def _hhmm(h):
    if h is None or (isinstance(h, float) and np.isnan(h)):
        return ''
    hh = int(h) % 24; mm = int(round((h % 1) * 60))
    if mm == 60: hh += 1; mm = 0
    return f"{hh:02d}:{mm:02d}"

def export_results(label, result, orders_df):
    import openpyxl
    from openpyxl.styles import Font, PatternFill, Alignment
    from openpyxl.utils import get_column_letter

    BLUE = "2E4057"; WHITE = "FFFFFF"; LGRAY = "F2F2F2"; AMBER = "FFC000"

    rows = []
    rnum = 1
    for day in DAYS:
        for route, res in zip(result['routes'].get(day, []),
                              result['results'].get(day, [])):
            if not res.get('feasible'): continue
            rows.append({'Route': rnum, 'Day': day, 'Stop': 'DC(start)',
                        'OrderID': 0, 'ZIP': DEPOT_ZIP, 'Cube_ft3': '',
                        'Arrival': '', 'Departure': _hhmm(res['departure_time']),
                        'CumDrive_h': '', 'CumDuty_h': '',
                        'RouteMiles': '', 'Overnight': ''})
            for stop in res.get('stops', []):
                cube = next((o['CUBE'] for o in route
                             if o['ORDERID'] == stop['order_id']), '')
                rows.append({'Route': rnum, 'Day': day, 'Stop': '',
                            'OrderID': stop['order_id'], 'ZIP': stop['zip'],
                            'Cube_ft3': cube,
                            'Arrival': _hhmm(stop['arrival']),
                            'Departure': _hhmm(stop['departure']),
                            'CumDrive_h': round(stop['drive_cumul'], 3),
                            'CumDuty_h': round(stop['duty_cumul'], 3),
                            'RouteMiles': '', 'Overnight': ''})
            rows.append({'Route': rnum, 'Day': day, 'Stop': 'DC(end)',
                        'OrderID': 0, 'ZIP': DEPOT_ZIP,
                        'Cube_ft3': sum(o['CUBE'] for o in route),
                        'Arrival': _hhmm(res['arrival_back_time']),
                        'Departure': '',
                        'CumDrive_h': round(res['drive_hrs'], 3),
                        'CumDuty_h': round(res['duty_hrs'], 3),
                        'RouteMiles': round(res['total_miles'], 1),
                        'Overnight': 'Yes' if res.get('is_overnight') else 'No'})
            rnum += 1

    csv_path = f"solution_{label}.csv"
    pd.DataFrame(rows).to_csv(csv_path, index=False)

    wb = openpyxl.Workbook()
    ws1 = wb.active
    ws1.title = "Day Summary"

    title = f"IE 7200 VRP Solution - {label.replace('_', ' ')}"
    ws1['A1'] = title
    ws1['A1'].font = Font(bold=True, size=14, color=WHITE)
    ws1['A1'].fill = PatternFill('solid', start_color=BLUE)
    ws1.merge_cells('A1:J1')

    hdrs = ['Day', 'Orders', 'Volume (ft3)', 'Routes', 'Overnight',
            'Total Miles', 'Avg Mi/Route', 'Feasible', 'Drive h (avg)',
            'Duty h (avg)']
    for c, h in enumerate(hdrs, 1):
        cell = ws1.cell(row=3, column=c, value=h)
        cell.font = Font(bold=True, color=WHITE)

```

```

cell.fill = PatternFill('solid', start_color=BLUE)
cell.alignment = Alignment(horizontal='center', wrap_text=True)
ws1.column_dimensions[get_column_letter(c)].width = 15

total_mi = 0.0
row = 4
for day in DAYS:
    routes = result['routes'].get(day, [])
    rr = result['results'].get(day, [])
    feas = [r for r in rr if r.get('feasible')]
    mi = sum(r['total_miles'] for r in feas)
    n_on = sum(1 for r in feas if r.get('is_overnight'))
    n_ord = sum(len(r) for r in routes)
    vol = sum(sum(o['CUBE'] for o in r) for r in routes)
    avg_mi = mi / len(feas) if feas else 0
    avg_dr = sum(r['drive_hrs'] for r in feas) / len(feas) if feas else 0
    avg_du = sum(r['duty_hrs'] for r in feas) / len(feas) if feas else 0
    total_mi += mi
    vals = [day, n_ord, int(vol), len(routes), n_on,
            round(mi,1), round(avg_mi,1), len(feas),
            round(avg_dr,2), round(avg_du,2)]
    for c, v in enumerate(vals, 1):
        cell = ws1.cell(row=row, column=c, value=v)
        cell.alignment = Alignment(horizontal='center')
        if row % 2 == 0:
            cell.fill = PatternFill('solid', start_color=LGRAY)
    row += 1

for c, v in enumerate(['TOTAL', '', '', '', '',
                      round(total_mi,1), '', '', '', ''], 1):
    cell = ws1.cell(row=row, column=c, value=v if v != '' else None)
    cell.font = Font(bold=True)
    cell.fill = PatternFill('solid', start_color=AMBER)

row += 2
for lbl, val in [('Annual miles (x52)', round(total_mi*52)),
                ('Min drivers', result['drivers']),
                ('Min vehicles', result['vehicles'])]:
    ws1.cell(row=row, column=1, value=lbl).font = Font(bold=True)
    ws1.cell(row=row, column=2, value=val)
    row += 1

ws2 = wb.create_sheet("Route Details")
hdrs2 = ['Route', 'Day', 'Stop', 'OrderID', 'ZIP', 'Cube_ft3',
         'Arrival', 'Departure', 'CumDrive_h', 'CumDuty_h', 'RouteMiles', 'Overnight']
for c, h in enumerate(hdrs2, 1):
    cell = ws2.cell(row=1, column=c, value=h)
    cell.font = Font(bold=True, color=WHITE)
    cell.fill = PatternFill('solid', start_color=BLUE)
    ws2.column_dimensions[get_column_letter(c)].width = 13

rn = 1; r2 = 2
for day in DAYS:
    for route, res in zip(result['routes'].get(day, []),
                        result['results'].get(day, [])):
        if not res.get('feasible'): continue
        def wr2(vals):
            nonlocal r2
            for c, v in enumerate(vals, 1):
                ws2.cell(row=r2, column=c, value=v)
            r2 += 1
        wr2([rn, day, 'DC(start)', 0, DEPOT_ZIP, '',
            '', _hhmm(res['departure_time']),
            '', '', '', ''])
        for stop in res.get('stops', []):
            cube = next((o['CUBE'] for o in route
                        if o['ORDERID'] == stop['order_id']), '')
            wr2([rn, day, '', stop['order_id'], stop['zip'], cube,
                _hhmm(stop['arrival']), _hhmm(stop['departure']),
                round(stop['drive_cumul'],3), round(stop['duty_cumul'],3),
                '', ''])
        wr2([rn, day, 'DC(end)', 0, DEPOT_ZIP,
            sum(o['CUBE'] for o in route),
            _hhmm(res['arrival_back_time']), '',
            round(res['drive_hrs'],3), round(res['duty_hrs'],3),
            round(res['total_miles'],1),
            'Yes' if res.get('is_overnight') else 'No'])

```

```

        rn += 1

    xlsx_path = f"solution_{label}.xlsx"
    wb.save(xlsx_path)
    print(f" => {csv_path} | {xlsx_path}")
    return csv_path, xlsx_path

def export_check_format(label, result):
    """Solution Check Tool column-B format: 0, order IDs..., 0, repeat."""
    rows = []
    for day in DAYS:
        for route in result['routes'].get(day, []):
            rows.append(0)
            for o in route:
                rows.append(o['ORDERID'])
            rows.append(0)
    path = f"check_tool_{label}.csv"
    pd.DataFrame({'OrderID_ColB': rows}).to_csv(path, index=False)
    print(f" => {path} (paste into Solution Check Tool column B)")

def main():
    t0 = time.time()
    print("Loading data...")
    orders, df_loc, dist_matrix = load_data()

    orders['TOZIP'] = pd.to_numeric(orders['TOZIP'], errors='coerce')
    orders.dropna(subset=['TOZIP'], inplace=True)
    orders['TOZIP'] = orders['TOZIP'].astype(int)

    df_loc['ZIP'] = pd.to_numeric(df_loc['ZIP'], errors='coerce')
    df_loc.dropna(subset=['ZIP'], inplace=True)
    df_loc['ZIP'] = df_loc['ZIP'].astype(int)

    print(f" {len(orders)} orders | {orders['DayOfWeek'].nunique()} days | "
          f"{orders['TOZIP'].nunique()} store zips")
    print(f" ST required: {(orders['ST required?']=='yes').sum()} orders | "
          f"Distance matrix: {dist_matrix.shape[0]}x{dist_matrix.shape[1]}")

    if DEPOT_ZIP not in df_loc['ZIP'].values:
        df_loc = pd.concat([df_loc, pd.DataFrame({'ZIP': [DEPOT_ZIP], 'X': [0], 'Y': [0]})], ignore_index=True)

    missing_zips = set(orders['TOZIP'].unique()) - set(df_loc['ZIP'].unique())
    if missing_zips:
        print(f" Warning: {len(missing_zips)} TOZIPs from orders are missing from LocationTable. Adding dummy coordinates.")
        dummy_df = pd.DataFrame({'ZIP': list(missing_zips), 'X': [0]*len(missing_zips), 'Y': [0]*len(missing_zips)})
        df_loc = pd.concat([df_loc, dummy_df], ignore_index=True)

    coords = df_loc.set_index('ZIP').apply(lambda row: (row['X'], row['Y']), axis=1).to_dict()

    q1 = solve_q1(orders, dist_matrix, verbose=True)
    export_results('Q1_BaseCase', q1, orders)
    export_check_format('Q1_BaseCase', q1)

    q2 = solve_q2(orders, dist_matrix, verbose=True)
    export_results('Q2_MixedFleet', q2, orders)
    export_check_format('Q2_MixedFleet', q2)

    q3 = solve_q3_best(orders, dist_matrix, coords, verbose=True, max_iter=25)
    export_results('Q3_Relaxed', q3, orders)
    export_check_format('Q3_Relaxed', q3)

    print(f"\nRuntime: {time.time()-t0:.1f}s")

    print(" FINAL COMPARISON")

    fmt = " {:<30s} {:>12s} {:>13s} {:>8s} {:>9s}"
    print(fmt.format('Scenario', 'Weekly Miles', 'Annual Miles', 'Drivers', 'Vehicles'))

    for lbl, q in [('Q1 Base Case (Van)', q1),
                  ('Q2 Mixed Fleet', q2),
                  ('Q3 Relaxed Schedule', q3)]:
        print(fmt.format(lbl, f"{q['weekly']:.1f}",
                        f"{q['annual']:.0f}",
                        str(q['drivers']), str(q['vehicles'])))
    for lbl, q in [('Q2 vs Q1', q2), ('Q3 vs Q1', q3)]:
        d = a['annual'] - a1['annual']

```

```

p = d / q1['annual'] * 100
sign = '+' if d >= 0 else '-'
print(f" {lbl}: {sign}{d:,.0f} mi/yr ({sign}{p:.1f}%)")

if __name__ == '__main__':
    main()

```

Loading data...

261 orders | 5 days | 123 store zips
ST required: 60 orders | Distance matrix: 123x123
Q1: Base Case - Van fleet, fixed delivery schedule
Mon: 43 orders, 10,223 ft3
5 routes | 1,361.0 mi | 0 overnight
Tue: 58 orders, 11,537 ft3
6 routes | 1,543.0 mi | 0 overnight
Wed: 50 orders, 15,192 ft3
6 routes | 1,196.0 mi | 0 overnight
Thu: 63 orders, 15,009 ft3
6 routes | 1,411.0 mi | 0 overnight
Fri: 47 orders, 13,468 ft3
5 routes | 1,203.0 mi | 0 overnight

Weekly miles : 6,714.0
Annual miles : 349,128
Min drivers : 6
Min vehicles : 6
=> solution_Q1_BaseCase.csv | solution_Q1_BaseCase.xlsx
=> check_tool_Q1_BaseCase.csv (paste into Solution Check Tool column B)

Q2: Mixed Fleet - Vans + Straight Trucks
Mon: 43 orders (ST=10, Van=33)
ST => 2 routes, 267.0 mi
Van => 4 routes, 1,318.0 mi
Tue: 58 orders (ST=16, Van=42)
ST => 3 routes, 321.0 mi
Van => 4 routes, 1,335.0 mi
Wed: 50 orders (ST=8, Van=42)
ST => 1 routes, 239.0 mi
Van => 6 routes, 1,153.0 mi
Thu: 63 orders (ST=18, Van=45)
ST => 3 routes, 391.0 mi
Van => 5 routes, 1,311.0 mi
Fri: 47 orders (ST=8, Van=39)
ST => 1 routes, 63.0 mi
Van => 5 routes, 1,186.0 mi

Weekly miles : 7,584.0
Annual miles : 394,368
Min drivers : 8
Min vehicles : 8
=> solution_Q2_MixedFleet.csv | solution_Q2_MixedFleet.xlsx
=> check_tool_Q2_MixedFleet.csv (paste into Solution Check Tool column B)

Q3: OPTIMAL GEOGRAPHIC CLUSTERING (A+ SOLUTION)
Running local improvement...
Running final VRP...
Mon: 89 orders → 9 routes, 1,674.0 miles
Tue: 54 orders → 5 routes, 1,350.0 miles
Wed: 33 orders → 3 routes, 475.0 miles
Thu: 68 orders → 6 routes, 649.0 miles
Fri: 17 orders → 3 routes, 645.0 miles

Weekly miles : 4,793.0
Annual miles : 249,236
Drivers : 9
Vehicles : 9
=> solution_Q3_Relaxed.csv | solution_Q3_Relaxed.xlsx
=> check_tool_Q3_Relaxed.csv (paste into Solution Check Tool column B)

